

1. Introduzione e obiettivo del sistema

FastFood è un'applicazione web per l'ordinazione online presso ristoranti di una catena di fast food. Il sistema deve gestire l'intero ciclo di vita di un ordine: dalla registrazione dell'utente, alla configurazione del ristorante e del menu, fino all'ordinazione, preparazione e consegna del cibo.

Essendo la specifica volutamente incompleta, in questo documento vengono esplicitate le interpretazioni adottate, motivandole, ogniqualvolta il testo lasci spazio ad ambiguità. Questo è coerente con quanto richiesto nella premessa del progetto ("il candidato deve essere in grado di valutare eventuali soluzioni alternative e giustificare le scelte implementative adottate").

2. Attori del Sistema

Attore	Descrizione
Cliente	Utente finale che si registra per ordinare cibo. Sfoglia ristoranti/piatti, gestisce carrello, effettua ordini, ne segue lo stato, riceve/segna la consegna.
Ristoratore	Utente che gestisce un ristorante: anagrafica, menu (piatti in vendita), coda di preparazione degli ordini, statistiche di vendita.
Sistema (attore non umano)	Componente automatico che calcola tempi di attesa in coda.

3. Requisiti funzionali

3.1 Macro-scenario 1 — Gestione del profilo utente

- **RF1.1** Il sistema deve permettere la registrazione di un nuovo utente, acquisendo almeno: username, email, password, tipologia utente (Cliente o Ristoratore).
- **RF1.2** In base alla tipologia scelta, il sistema deve acquisire dati aggiuntivi specifici:
 - *Cliente*: nome, cognome, metodo di pagamento (carta di credito/prepagata), preferenze di prodotto (per notifiche/offerte in bacheca).
 - *Ristoratore*: nome ristorante, telefono, partita IVA, indirizzo.
- **RF1.3** Il sistema deve permettere il login tramite credenziali (username/email + password).

- **RF1.4** Il sistema deve permettere la modifica dei dati personali/preferenze, sia per Cliente sia per Ristoratore.
- **RF1.5** Il sistema deve permettere la cancellazione del proprio profilo.

Assunzione: un ristoratore è associato a **un solo ristorante** (relazione 1:1), poiché la specifica non menziona la possibilità di gestire più punti vendita per lo stesso utente ristoratore.

3.2 Macro-scenario 2 — Gestione del ristorante

- **RF2.1** Il ristoratore, autenticato, deve poter inserire/modificare i dati del proprio ristorante (nome, luogo, telefono, P.IVA, indirizzo).
- **RF2.2** Il ristoratore deve poter comporre il proprio menu selezionando piatti da una lista comune fornita da meal.json (dati precaricati al setup).
- **RF2.3** Il ristoratore deve poter inserire **piatti personalizzati** (non presenti nella lista comune), specificando per ognuno tutte le informazioni previste: nome, tipologia, prezzo, ingredienti, foto illustrativa.
- **RF2.4** Per ogni piatto (di listino comune o personalizzato) il sistema deve gestire: nome, tipologia, prezzo, elenco ingredienti, immagine.
- **RF2.5** Il ristoratore deve poter modificare/rimuovere piatti dal proprio menu.

3.3 Macro-scenario 3 — Gestione degli ordini

- **RF3.1** Il cliente autenticato deve poter sfogliare i ristoranti e i relativi menu.
- **RF3.2** Il cliente deve poter aggiungere uno o più piatti al carrello, anche di ristoranti diversi (da chiarire, vedi ambiguità sotto) o vincolando il carrello a un solo ristorante per ordine.
- **RF3.3** Il cliente deve poter concludere l'acquisto (checkout) generando un ordine.
- **RF3.4** Ogni ordine deve seguire il flusso di stato:

ordinato → in preparazione → consegnato

- **RF3.5** Ritiro presso il ristorante: il sistema calcola un tempo di attesa stimato basato sugli ordini già in coda presso quel ristorante. Quando il ristoratore segnala che l'ordine è pronto, l'ordine passa **direttamente** allo stato "consegnato" (bypassando "in consegna") e viene rimosso dalla coda di preparazione.
- **RF3.6** Il ristoratore deve poter visualizzare la coda degli ordini del proprio ristorante e segnalarne il completamento della preparazione.

3.4 Funzionalità aggiuntive richieste dalla versione Full

- **RF5.1** Visualizzazione delle informazioni su piatti, clienti, ristoratori registrati e acquisti.
 - **RF5.2** Ricerca ristoranti per: luogo, nome.
 - **RF5.3** Ricerca piatti per: tipologia, nome, prezzo.
 - **RF5.8** Visualizzazione storico acquisti (presenti e passati) per un cliente.
-

4. Requisiti non funzionali

- **RNF1 — Architettura:** separazione netta tra frontend (HTML5/CSS3/JS) e backend (Node.js + MongoDB), comunicanti tramite API REST.
- **RNF2 — Documentazione API:** le API REST devono essere documentate tramite Swagger (OpenAPI).
- **RNF3 — Separazione struttura/presentazione:** le pagine HTML5 non devono contenere stili inline; tutto lo stile va demandato ai fogli CSS3.
- **RNF4 — Persistenza:** tutti i dati (utenti, ristoranti, piatti, ordini) devono essere persistiti in MongoDB.
- **RNF5 — Setup iniziale:** all'avvio dell'applicazione, i dati di base (lista piatti comuni da meal.json) devono essere caricati.
- **RNF7 — Usabilità:** interfaccia semplice e coerente per i due profili utente, con viste distinte per Cliente e Ristoratore.

5. Entità principali (modello dati preliminare)

Entità	Attributi principali
Utente	id, username, email, password, tipo (Cliente/Ristoratore)
Cliente	<i>(estende Utente)</i> nome, cognome, metodo di pagamento, preferenze
Ristoratore	<i>(estende Utente)</i> nome ristorante, telefono, P.IVA, indirizzo, luogo
Piatto	id, nome, tipologia, prezzo, ingredienti[], foto, id ristorante
Ordine	id, id cliente, id ristorante, lista piatti/quantità, stato, timestamp
Carrello	id cliente, lista piatti/quantità (temporaneo, pre-ordine)

6. Elenco sintetico dei casi d'uso principali

Cliente:

UC1 Registrazione • UC2 Login • UC3 Modifica profilo • UC4 Cancellazione profilo • UC5 Ricerca ristoranti • UC6 Ricerca piatti (per nome/tipo/prezzo) • UC7 Aggiunta piatto al carrello • UC8 Checkout ordine • UC9 Visualizzazione stato ordine • UC10 Visualizzazione storico ordini

Ristoratore:

UC12 Registrazione • UC13 Login • UC14 Modifica profilo/ristorante • UC15 Cancellazione profilo • UC16 Composizione menu (da listino comune) • UC17 Creazione piatto personalizzato • UC18 Modifica/rimozione piatto • UC19 Visualizzazione coda ordini • UC20 Segnalazione ordine pronto • UC21 Visualizzazione statistiche ristorante

Sistema:

Caricamento iniziale dati da meal.json • Calcolo tempi di attesa ordini
